



Secure Packages with CodeArtifact



Arpita Chowdhury

Packages

Info

🔄

Delete package

View connection instructions

🔍

Filter by package name prefix, format, namespace prefix, and origin controls

<

1

2

3

4

>

⚙️

	Package name	Namespace	Format	Latest version	Latest publish date	Publish	Upstream
☐	backport-util-concurrent	backport-util-concurrent	maven	3.1	1 minute ago	Block	Allow
☐	classworlds	classworlds	maven	1.1	2 minutes ago	Block	Allow
☐	google	com.google	maven	1	1 minute ago	Block	Allow
☐	jsr305	com.google.code.findbugs	maven	2.0.1	1 minute ago	Block	Allow
☐	google-collections	com.google.collections	maven	1.0	1 minute ago	Block	Allow
☐	commons-cli	commons-cli	maven	1.0	2 minutes ago	Block	Allow
☐	commons-logging-api	commons-logging	maven	1.1	1 minute ago	Block	Allow
☐	junit	junit	maven	3.8.2	1 minute ago	Block	Allow
☐	log4j	log4j	maven	1.2.12	1 minute ago	Block	Allow
☐	apache	org.apache	maven	5	1 minute ago	Block	Allow
☐	maven	org.apache.maven	maven	2.2.1	1 minute ago	Block	Allow
☐	maven-artifact	org.apache.maven	maven	2.2.1	1 minute ago	Block	Allow



Introducing Today's Project!

In this project, I will demonstrate setting up a Java web application on Amazon EC2, connecting the project to a GitHub repository, and creating an AWS CodeArtifact repository with the required permissions so my EC2 instance can securely store and retrieve application packages. I'm doing this project to learn how modern DevOps workflows manage code, dependencies, and access control in the cloud, and how these components come together as foundational building blocks of a secure, automated CI/CD pipeline.

Key tools and concepts

Services I used were AWS CodeArtifact, AWS Identity and Access Management (IAM), Amazon EC2, and the AWS CLI to create, publish, secure, and retrieve a custom package in a private repository. Key concepts I learnt include private package repositories, package versioning, artifact integrity verification using SHA-256 hashes, authentication with temporary authorization tokens, upstream and origin controls, and the complete package lifecycle from publishing to consumption in a real-world DevOps workflow.



Project reflection

This project took me approximately 4 hours to complete, including time spent troubleshooting multiple real-world issues. The most challenging part was dealing with repeated errors end-to-end: CodeArtifact authentication failures (401 Unauthorized), Maven dependency/plugin resolution issues due to missing configuration, token expiration and environment variable setup, and my SSH connection randomly dropping, which disrupted my workflow and forced me to reconnect and rerun commands multiple times. To fix the SSH drops, I updated my SSH configuration with keep-alive settings (e.g., `ServerAliveInterval` and `ServerAliveCountMax`) so the connection stayed active during idle periods and long builds. It was most rewarding to finally publish my own custom package to CodeArtifact, download it back, and validate the contents successfully because it proved I can troubleshoot production-style DevOps problems (auth + build tooling + connectivity) and still deliver working results.

This project is part three of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project Continuous Integration with CodeBuild



CodeArtifact Repository

CodeArtifact is an AWS-managed artifact repository service that securely stores, publishes, and retrieves software packages such as Java (Maven), npm, Python, and other dependencies used during application builds. Engineering teams use artifact repositories because they provide a centralized and trusted location to manage dependencies, improve build consistency, enhance security by controlling package access, reduce reliance on public repositories, and enable faster, more reliable CI/CD pipelines by caching and reusing approved artifacts.

A domain is the top-level organizational boundary in AWS CodeArtifact that groups repositories together and centrally manages security, permissions, and access policies across them. My domain is nextwork.

A CodeArtifact repository can have an upstream repository, which means it can automatically fetch packages from another repository when the package is not already stored locally, and then cache those packages for future use. My repository's upstream repository is Maven Central, which allows my CodeArtifact repository to pull Java dependencies from the public Maven ecosystem when needed and store them securely inside AWS for faster, more reliable future builds.



Review and create Info

Package flow

Review how packages flow from external connections through nextwork to nextwork-devops-cicd.

External connections

Domain: **nextwork**

public:maven-central



Upstream repository ?
maven-central-store



Repository
nextwork-devops-cicd



CodeArtifact Security

Issue

To access CodeArtifact, we need valid AWS credentials (or an IAM role attached to the EC2 instance) with permission to call CodeArtifact APIs and retrieve an authorization token. I ran into an error when retrieving a token because my EC2 instance did not have AWS credentials configured (no IAM role/instance profile attached), so the AWS CLI couldn't authenticate and returned "Unable to locate credentials."

Resolution

To resolve the error with my security token, I assigned an IAM role to my EC2 instance that granted access to AWS CodeArtifact. This resolved the error because AWS automatically supplied secure, short-lived credentials, enabling successful authorization token retrieval.

It's a security best practice to use IAM roles because they enable secure, short-lived credential access that is automatically managed by AWS, minimizing exposure, improving auditability, and enforcing least-privilege access.



The JSON policy attached to my role

The JSON policy I set up grants permissions for my EC2 instance to retrieve CodeArtifact authorization tokens, access repository endpoints, and read packages using temporary credentials provided by AWS STS.

Policy editor

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "codeartifact:GetAuthorizationToken",
8         "codeartifact:GetRepositoryEndpoint",
9         "codeartifact:ReadFromRepository"
10      ],
11       "Resource": "*"
12     },
13     {
14       "Effect": "Allow",
15       "Action": "sts:GetServiceBearerToken",
16       "Resource": "*",
17       "Condition": {
18         "StringEquals": {
19           "sts:AWSServiceName": "codeartifact.amazonaws.com"
20         }
21       }
22     }
23   ]
24 }
25 |
```



Maven and CodeArtifact

To test the connection between Maven and CodeArtifact, I compiled my web app using settings.xml

The settings.xml file configures Maven to use AWS CodeArtifact as the central artifact repository by defining authentication credentials, activating a CodeArtifact profile by default, and mirroring all dependency requests through the CodeArtifact Maven endpoint.

Compiling means converting human-written source code into a machine-readable form (bytecode or binaries) while checking the code for errors so the program can run or be packaged.



Arpita Chowdhury

NextWork Student

nextwork.org

```
settings.xml  Extension: Extension Pack for Java  Welcome
1  <settings>
2  <servers>
3  <server>
4    <id>nextwork-nextwork-devops-cicd</id>
5    <username>aws</username>
6    <password>${env.CODEARTIFACT_AUTH_TOKEN}</password>
7  </server>
8 </servers>
9 <profiles>
10 <profile>
11   <id>nextwork-nextwork-devops-cicd</id>
12   <activation>
13     <activeByDefault>true</activeByDefault>
14   </activation>
15   <repositories>
16     <repository>
17       <id>nextwork-nextwork-devops-cicd</id>
18       <url>https://nextwork-006834263464.d.codeartifact.us-east-1.amazonaws.com/maven/nextw
19     </repository>
20   </repositories>
21 </profile>
22 </profiles>
23 <mirrors>
24 <mirror>
25   <id>nextwork-nextwork-devops-cicd</id>
26   <name>nextwork-nextwork-devops-cicd</name>
27   <url>https://nextwork-006834263464.d.codeartifact.us-east-1.amazonaws.com/maven/nextwork-
28   <mirrorOf>*</mirrorOf>
29 </mirror>
30 </mirrors>
31 </settings>
32
```



Verify Connection

After compiling, I checked the AWS CodeArtifact repository. I noticed that multiple Maven packages and plugins (such as Apache, JUnit, Log4j, and Maven artifacts) appeared with recent publish timestamps, confirming that dependencies and plugins were successfully pulled and cached in the repository during the compilation process.

Packages info

🔄

Delete package

View connection instructions

🔍

Filter by package name prefix, format, namespace prefix, and origin controls

<

1

2

3

4

>

⚙

	Package name	Namespace	Format	Latest version	Latest publish date	Publish	Upstream
☐	backport-util-concurrent	backport-util-concurrent	maven	3.1	1 minute ago	Block	Allow
☐	classworlds	classworlds	maven	1.1	2 minutes ago	Block	Allow
☐	google	com.google	maven	1	1 minute ago	Block	Allow
☐	jsr305	com.google.code.findbugs	maven	2.0.1	1 minute ago	Block	Allow
☐	google-collections	com.google.collections	maven	1.0	1 minute ago	Block	Allow
☐	commons-cli	commons-cli	maven	1.0	2 minutes ago	Block	Allow
☐	commons-logging-api	commons-logging	maven	1.1	1 minute ago	Block	Allow
☐	junit	junit	maven	3.8.2	1 minute ago	Block	Allow
☐	log4j	log4j	maven	1.2.12	1 minute ago	Block	Allow
☐	apache	org.apache	maven	5	1 minute ago	Block	Allow
☐	maven	org.apache.maven	maven	2.2.1	1 minute ago	Block	Allow
☐	maven-artifact	org.apache.maven	maven	2.2.1	1 minute ago	Block	Allow



Uploading My Own Packages

In a project extension, I also decided to publish my own custom Maven package to AWS CodeArtifact and then download it back as a dependency. This is useful in situations where a team needs to share private, reusable code (like internal utilities, shared validation logic, logging wrappers, or company-specific SDKs) across multiple projects without exposing it publicly on Maven Central.

To create my own package, I wrote a small custom file (`secret-mission.txt`) and bundled it into a compressed archive (`secret-mission.tar.gz`) so it could be uploaded and managed as a versioned package asset in CodeArtifact. I also generated a security hash because the SHA-256 checksum acts like a fingerprint for the file, letting me verify the package wasn't corrupted or tampered with during upload/download and ensuring I'm always retrieving the exact same artifact.

To publish the package, I uploaded my custom archive to AWS CodeArtifact as a new internal package version, making it available in my private repository just like a real production artifact. When I look at the package details in CodeArtifact, I can see my custom package (`secret-mission`) successfully published with version `1.0.0`, marked as internal, with a recent publish timestamp, confirming that the package lifecycle from creation to publishing was completed successfully.



To validate my package, I downloaded the published asset directly from AWS CodeArtifact and extracted it locally to confirm it could be retrieved and used like a real dependency. I saw the original file contents (“Helloooooo this is a test package!”) exactly as expected, which verified that the package was stored correctly, downloaded successfully, and remained unchanged throughout the publish-and-retrieve lifecycle.

```
{
  "format": "generic",
  "namespace": "secret-mission",
  "package": "secret-mission",
  "version": "1.0.0",
  "versionRevision": "pg7UjvpYRAY15iodWpP7rZ73pBwhQwZ2sahMERLpuhc=",
  "status": "Published",
  "asset": {
    "name": "secret-mission.tar.gz",
    "size": 168,
    "hashes": {
      "MD5": "008cffa3d249e1ca93c530df4d110f59",
      "SHA-1": "a5dc9320b080e48db6bbca10048d14e04bbcc36c",
      "SHA-256": "cfaa17d4adf2b565c7940f4b9a25a440988ce5dfc641a81be7d1517c6efdf20a",
      "SHA-512": "64907e3decbb6434bc38ce9bafae49b181741bd9ebea01b227999b7b8a2e8d548f4c618e73fac6f5bd5aad67c89eb0f7392db67deecac677Faca510092a37002"
    }
  }
}
```




nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

